# Nested Loops

## Loop Inside Another Loop

*Same Loops. Bigger Possibilities.*

Think
Code
Practice
Solve
Grow

## What are Nested Loops?

A nested loop is a loop inside another loop. The **inner loop** runs completely for each iteration of the **outer loop**.

## Syntax

```
for i in range(n):      # outer loop
    for j in range(m):  # inner loop
        # statements
```

## Example: Print a Square Pattern

Write a Python program using nested loops to print the following pattern for `n = 5`.

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

## Python Code

```
1  n = 5
2  for i in range(n):
3      for j in range(n):
4          print("*", end=" ")
5      print()
```

## Output

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

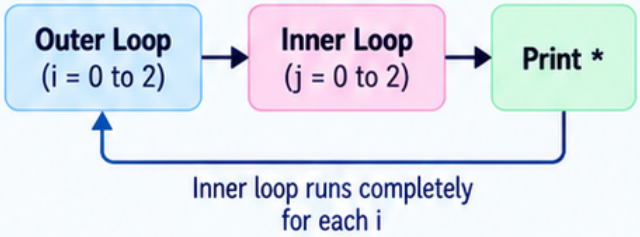
## How It Works?

1. The outer loop (i) runs from 0 to n-1 and controls the number of rows.
2. The inner loop (j) runs from 0 to n-1 and prints * in each column.
3. After the inner loop completes, we move to the next line using `print()`
4. This creates a **n × n** square pattern.

## Key Takeaways

- ✅ Nested loop = loop inside another loop.
- ✅ The inner loop runs completely for each iteration of the outer loop.
- ✅ Useful for patterns, matrices, tables, and 2D data.
- ✅ Practice different patterns (square, rectangle, triangle, number patterns, etc.).
- ✅ Proper indentation makes the code readable.

### Visual Representation (n = 3)




"Small loops solve small problems, nested loops solve bigger ones."

– Satyarthi

python
Learn | Practice | Grow

# Nested Loops

## Practice Question

*Same Loops. Greater Possibilities.*

Think
Code
Practice
Improve
Be a Better You

## 1 Question

Write a Python program using nested loops to print the following pattern for a given number of rows `n`.
(Take n as input from the user)

### Pattern (n = 4):

```
1
1 2
1 2 3
1 2 3 4
```

## </> Python Code

```python
n = int(input("Enter n: "))

for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

## Sample Run

```
Enter n: 4
1
1 2
1 2 3
1 2 3 4
```

## How It Works?

1. Take n as input from the user.
2. The outer loop (i) runs from 1 to n and controls the number of rows.
3. The inner loop (j) runs from 1 to i and prints numbers from 1 to i.
4. After each row, move to the next line using `print()`
5. This creates a number pattern.

## Key Takeaways

- ☑ Nested loops are used to solve pattern-based problems.
- ☑ The inner loop runs completely for each iteration of the outer loop.
- ☑ Useful for patterns, matrices, tables and other 2D problems.
- ☑ Helps in building logical thinking and problem-solving skills.
- ☑ Practice more patterns to strengthen your concepts.

"Practice loops today, solve bigger problems tomorrow." – Satyarthi

python
Learn | Practice | Grow

# Nested Loops
## Practice Question

*Same Loops. Greater Possibilities.*

Think
Code
Practice
Solve
Be a Better You ☺

## 1 Question

Write a Python program using nested loops to multiply two matrices of size `n × m` and `m × p`.
Take all elements as input from the user and print the resulting matrix.

## Example

If $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

Then $C = A \times B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

## </> Python Code

```
1  # Matrix multiplication using nested loops
2  n = int(input("Enter number of rows of matrix A: "))
3  m = int(input("Enter number of columns of matrix A
           (and rows of B): "))
4  p = int(input("Enter number of columns of matrix B: "))
5
6  print("Enter elements of matrix A:")
7  A = [[0] * m for _ in range(n)]
8  for i in range(n):
9      for j in range(m):
10         A[i][j] = int(input())
11
12 print("Enter elements of matrix B:")
13 A = [[0] * m for _ in range(m)]
14 for i in range(m):
15     for j in range(p):
16         B[i][j] = int(input())
17
18 # Result matrix
19 C = [[0] * p for _ in range(n)]
20 for i in range(n):
21     for j in range(p):
22         for k in range(m):
22             C[i][j] += A[i][k] * B[k][j]
24
25 print("Resultant Matrix (A x B):")
26 for i in range(n):
27     for j in range(p):
28         print(C[i][j], end=" ")
29     print()
```

## Sample Run

```
Enter number of rows of matrix A: 2
Enter number of columns of matrix A
(and rows of B): 2
Enter number of columns of matrix B: 2
Enter elements of matrix A:
1 2
3 4
Enter elements of matrix B:
5 6
7 8

Resultant Matrix (A x B):
19 22
43 50
```

## How It Works?

1. Take matrix dimensions and all elements as input.
2. Use three nested loops (i, j, k).
3. For each element C[i][j], multiply the corresponding row of A with the column of B.
4. Store the result in matrix C.
5. Finally, print the resulting matrix.

## Key Takeaways

- ☑ Nested loops are useful for matrix operations.
- ☑ The inner loop runs completely for each iteration of the outer loop.
- ☑ Helps in solving real-world problems like data processing, image operations, etc.
- ☑ Understand the logic with smaller examples first.
- ☑ Use meaningful variable names and proper indentation.
- ☑ Practice different problems (addition, subtraction, transpose, etc.).
- ☑ Nested loops build stronger problem-solving skills.

*"Practice today, so you can build bigger solutions tomorrow."*

– Satyarthi ☺

python
Learn | Practice | Grow

# Nested Loops
## Practice Question

*Same Loops. Greater Possibilities.*

Think
Code
Practice
Solve
Be a Better You

## 1 Question

Write a Python program using nested loops to print the following number pattern for a given number of rows `n`.
(Take n as input from the user)

## Pattern (n = 5):

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

## </> Python Code

```
1  n = int(input("Enter number of rows: "))
2
3  for i in range(1, n + 1):    # outer loop
4    for j in range(1, i + 1):  # inner loop
5      print(j, end=" ")
6    print()
8
```

## Sample Run

```
Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

## How It Works?

1. Take n as input from the user.
2. The outer loop (i) runs from 1 to n and controls the number of rows.
3. The inner loop (j) runs from 1 to i and prints numbers from 1 to i.
4. After each row, move to the next line using `print()`
5. This creates an increasing number triangle.

## Visual Representation (n = 4)

| | | |
|---|---|---|
| i = 1 | 1 | → Inner loop runs 1 time |
| i = 2 | 1 2 | → Inner loop runs 2 times |
| i = 3 | 1 2 3 | → Inner loop runs 3 times |
| i = 4 | 1 2 3 4 | → Inner loop runs 4 times |

In each row i, we print numbers from 1 to i.

## Key Takeaways

- ☑ Nested loops are useful for pattern-based problems.
- ☑ The inner loop runs completely for each iteration of the outer loop.
- ☑ Helps in building logical thinking and problem-solving skills.
- ☑ Can be used to print different patterns (pyramid, inverted, number patterns, etc.).
- ☑ Use meaningful variable names and proper indentation.
- ☑ Practice more problems to strengthen your concepts.

*"Patterns are everywhere, you just need the right loops to find them."* – Satyarthi

python
Learn | Practice | Grow

# Python Lists

Store • Access • Modify • Explore

*One List. Many Possibilities.*

Write
Better Code
Build
Bigger Dreams !

## 1 What is a List?

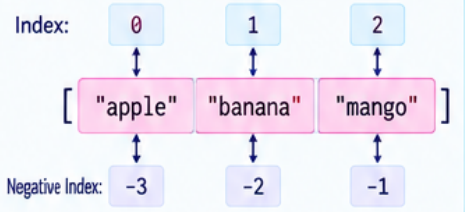
A list is a built-in data type in Python used to store multiple items in a single variable. Lists can store different data types, are **ordered, mutable** (can be changed), and allow **duplicate values**.

## 2 Creating a List

```
# List of numbers
nums = [1, 2, 3, 4, 5]
# List of strings
fruits = ["apple", "banana", "mango"]
# Mixed data types
mixed = [1, "hello", 3.14, True]
```

## 3 Accessing Elements

```
fruits = ["apple", "banana", "mango"]
print(fruits[0])   # apple
print(fruits[1])   # banana
print(fruits[-1])  # mango (last element)
```




## 4 Modifying a List

```
fruits = ["apple", "banana", "mango"]
fruits[1] = "orange"   # change item
fruits.append("grapes") # add at end
fruits.insert(1, "kiwi") # add at index
fruits.remove("mango")  # remove item
fruits.pop()            # remove last item
```

**Final List:**

```
print(fruits)
# ['apple', 'kiwi', 'orange', 'grapes']
```

## 5 Useful List Functions

- `len(list)` → Number of items
- `list.append(x)` → Add item at end
- `list.insert(i, x)` → Add item at index
- `list.remove(x)` → Remove first occurrence
- `list.pop(i)` → Remove item at index
- `list.sort()` → Sort the list
- `list.reverse()` → Reverse the list
- `list.count(x)` → Count occurrences
- `list.index(x)` → Return index of item

## 6 Looping Through a List

```
fruits = ["apple","banana","mango"]
# Using for loop
for fruit in fruits:
    print(fruit)
```

## 7 List Comprehension

```
# Create a list of squares
squares = [x**2 for x in range(1, 6)]
print(squares)
# [1, 4, 9, 16, 25]
```

## 8 Slicing a List

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])   # [20, 30, 40]
print(nums[:3])    # [10, 20, 30]
print(nums[::2])   # [10, 30, 50]
```

## Example: Find Even Numbers from a List

Write a Python program to take a list of numbers from the user and print a new list containing only the even numbers.

```
# Input list
nums = list(map(int, input("Enter numbers (space separated): ").split()))
# Using list comprehension
even_nums = [x for x in nums if x % 2 == 0]
print("Even numbers:", even_nums)
```

## Sample Run

```
Enter numbers (space separated):
1 2 3 4 5 6
Even numbers: [2, 4, 6]
```

## Key Takeaways

- ✅ Lists can store multiple items of different data types.
- ✅ Lists are ordered, mutable, and allow duplicate values.
- ✅ Use indexing, slicing, and loops to access elements.
- ✅ Many built-in functions make list operations easy.
- ✅ Lists are widely used in real-world applications.

## Real-World Uses

- ✅ Storing student marks
- ✅ Managing to-do lists
- ✅ Processing data (e.g., sensor readings)
- ✅ Building menus in applications
- ✅ Used in data analysis, machine learning, and more

*"Master the basics today, create amazing things tomorrow."* – Satyarthi

python
Learn | Practice | Grow

# Python List Methods

## Built-in Functions to Work with Lists

*Simple Methods. Powerful Possibilities.*

Practice
Experiment
Build
Be Better !

## What are List Methods?

List methods are built-in functions in Python that help you perform common operations on lists, such as **adding**, **removing**, **modifying**, **searching**, and more.
They make working with lists easier and more efficient.

## Common List Methods

- append()
- extend()
- insert()
- remove()
- pop()
- clear()
- index()
- count()
- sort()
- reverse()
- copy()
- len()
- min()
- max()

### 1 append()

Adds an item at the end of the list.

```
fruits = ["apple", "banana"]
fruits.append("mango")
print(fruits)
```

**Output:** ['apple', 'banana', 'mango']

### 2 extend()

Adds all items from another list at the end.

```
num = [1, 2]
num.extend([3, 4, 5])
print(num)
```

**Output:** [1, 2, 3, 4, 5]

### 3 insert()

Adds an item at a specific index.

```
nums = [10, 20, 40]
nums.insert(2, 30)
print(nums)
```

**Output:** [10, 20, 30, 40]

### 4 remove()

Removes the first occurrence of the specified item.

```
fruits = ["apple", "banana", "mango"]
fruits.remove("banana")
print(fruits)
```

**Output:** ['apple', 'mango']

### 5 pop()

Removes and returns item at a given index (default: last).

```
nums = [1, 2, 3, 4]
removed = nums.pop(2)
print(removed)
print(nums)
```

**Output:** 3
[1, 2, 4]

### 6 clear()

Removes all items from the list.

```
nums = [1, 2, 3]
nums.clear()
print(nums)
```

**Output:** []

### 7 index()

Returns the index of the first occurrence of an item.

```
fruits = ["apple", "banana", "mango"]
print(fruits.index("mango"))
```

**Output:** 2

### 8 count()

Returns the number of times an item appears.

```
nums = [1, 2, 2, 3, 2]
print(nums.count(2))
```

**Output:** 3

### 9 sort()

Sorts the list in ascending order (modifies the original list).

```
nums = [4, 1, 3, 2]
nums.sort()
print(nums)
```

**Output:** [1, 2, 3, 4]

### 10 reverse()

Reverses the list order (modifies the original list).

```
nums = [1, 2, 3, 4]
nums.reverse()
print(nums)
```

**Output:** [4, 3, 2, 1]

### 11 copy()

Returns a shallow copy of the list.

```
a = [1, 2, 3]
b = a.copy()
print(b)
```

**Output:** [1, 2, 3]

### 12 len()

Returns the number of items in the list.

```
nums = [10, 20, 30, 40]
print(len(nums))
```

**Output:** 4

## Key Takeaways

- ✓ List methods help in performing common operations easily.
- ✓ Some methods modify the original list (e.g., sort(), append()).
- ✓ Some methods return a value (e.g., pop(), index(), count()).
- ✓ Lists are mutable, so you can change them anytime.
- ✓ Practice these methods to solve real-world problems.

## Real-World Uses

- ✓ Managing student lists
- ✓ Storing and updating data (e.g., inventory, marks)
- ✓ Processing data (e.g., sensor readings, logs)
- ✓ Building menus and user input systems
- ✓ Used in data analysis and machine learning

Code | Practice | Grow
A Better Tomorrow

*"Small methods make big programs."*

– Satyarthi

python
Learn | Practice | Grow

# Python Lists

## List Methods - Practice Question

*Practice Today. Stronger Tomorrow.*

Think
Code
Practice
Improve
Be a Better You

## 1 Question



Write a Python program that:

1. Takes a list of integers from the user (space separated).
2. Adds a new number to the list using `append()`.
3. Removes the first occurrence of a specific number using `remove()`.
4. Sorts the list in ascending order using `sort()`.
5. Finally, prints the updated list.

"Same methods. New possibilities. Your turn!"

## Python Code

```python
# Take list input from user
numbers = list(map(int, input("Enter numbers: ").split()))

# Add a new number
new_num = int(input("Enter a number to add: "))
numbers.append(new_num)

# Remove a specific number (first occurrence)
rem_num = int(input("Enter a number to remove: "))
numbers.remove(rem_num)

# Sort the list
numbers.sort()

# Print the updated list
print("Updated list:", numbers)
```

## Sample Run

```
Enter numbers: 5 3 8 3 1
Enter a number to add: 7
Enter a number to remove: 3
Updated list: [1, 3, 5, 7, 8]
```

## Explanation

1. We take a list of integers as input using `list()` and `map()`.
2. `append()` adds the new number at the end of the list.
3. `remove()` removes the first occurrence of the specified number.
4. `sort()` arranges the list in ascending order.
5. Finally, we print the updated list.

## Key Takeaways

- ☑ Lists are mutable (can be changed).
- ☑ List methods like append(), remove(), and sort() make list operations easy.
- ☑ These methods are widely used in real-world programs.
- ☑ Understanding list methods helps in data handling and problem solving.

## Try It Yourself

Modify the program to:

- ☑ Use sort(reverse=True) to sort in descending order.
- ☑ Remove all occurrences of a number.
- ☑ Count how many times a number appears before removing it.

*"Small practice steps lead to big programming skills."* – Satyarthi

python
Learn | Practice | Grow

# Python Lists

## Practice Question 2

*Apply List Methods. Solve Real Problems.*

Practice
Code
Build Logic
Improve
Be a Better You

## 1 Question

Write a Python program to:

1. Take a list of marks (integers) from the user (space separated).
2. Add a new mark using `append()`.
3. Remove the lowest mark using `remove()`.
4. Sort the list in descending order using `sort(reverse=True)`.
5. Print the final list and also the highest mark (first element).




*Work with lists, solve real problems!*

## </> Python Code

```
# Take input list of marks
marks = list(map(int, input("Enter marks (space
        separated): ").split()))

# Add a new mark
new_mark = int(input("Enter a new mark to add: "))
marks.append(new_mark)

# Remove the lowest mark
lowest = min(marks)
marks.remove(lowest)

# Sort in descending order
marks.sort(reverse=True)

# Print results
print("Final list of marks:", marks)
print("Highest mark:", marks[0])
```

## Sample Run

```
Enter marks (space separated): 78 92 65 88 70
Enter a new mark to add: 95
Final list of marks: [95, 92, 88, 78, 70]
Highest mark: 95
```

## Explanation

1. We take a list of integers using `list()` and `map()`.
2. We add a new mark using `append()`.
3. We find and remove the lowest mark using `min()` and `remove()`.
4. We sort the list in descending order using `sort(reverse=True)`.
5. Finally, we print the updated list and the highest mark (which is the first element).

## Key Takeaways

- ☑ Lists can store multiple values.
- ☑ Methods like append(), remove(), and sort() help manipulate data easily.
- ☑ min() and max() are useful to find smallest/largest values.
- ☑ Combining multiple list methods solves real-life problems.
- ☑ Practice helps in building logic and confidence.

## Try It Yourself

- ☑ Modify the program to also remove the highest mark instead of the lowest.
- ☑ Find the average of the final list.
- ☑ Allow the user to add multiple new marks.
- ☑ Use sorted(marks, reverse=True) without modifying the original list.

Code | Practice | Grow
A Better Tomorrow

*"Practice problems today, smarter solutions tomorrow."* – Satyarthi

python
Learn | Practice | Grow

# Python Lists

## Practice Question 3

*Use List Methods to Solve a Real Problem.*

Practice
Code
Think Logic
Build Skills
Be a Better You

## 1 Question

Write a Python program to manage a `to-do list` using list methods.
The program should:

1. Start with an empty list.
2. Allow the user to add tasks (use `append()`).
3. Allow the user to remove a task (use `remove()`).
4. Display the current list of tasks.
5. Finally, show the total number of tasks (use `len()`).




*Small programs create big habits!*

## </> Python Code

```python
# To-Do List using list methods
tasks = []
while True:
    print("\n--- To-Do List Menu ---")
    print("1. Add Task")
    print("2. Remove Task")
    print("3. Show Tasks")
    print("4. Total Tasks")
    print("5. Exit")
    choice = input("Enter your choice: ")
    if choice == '1':
        task = input("Enter a task: ")
        tasks.append(task)
        print("Task added!")
    elif choice == '2':
        task = input("Enter task to remove: ")
        if task in tasks:
            tasks.remove(task)
            print("Task removed!")
        else:
            print("Task not found in the list.")
    elif choice == '3':
        print("Current Tasks:", tasks)
    elif choice == '4':
        print("Total Tasks:", len(tasks))
    elif choice == '5':
        print("Goodbye!")
        break
    else:
        print("Invalid choice! Try again.")
```

## Sample Run

```
--- To-Do List Menu ---
1. Add Task
2. Remove Task
3. Show Tasks
4. Total Tasks
5. Exit
Enter your choice: 1
Enter a task: Study Physics
Task added!

Enter your choice: 1
Enter a task: Read a book
Task added!

Enter your choice: 3
Current Tasks: ['Study Physics', 'Read a book']

Enter your choice: 4
Total Tasks: 2

Enter your choice: 2
Enter task to remove: Read a book
Task removed!

Enter your choice: 3
Current Tasks: ['Study Physics']

Enter your choice: 5
Goodbye!
```

## Key Takeaways

- Lists are mutable, so you can add and remove items easily.
- Methods like append(), remove(), and len() are very useful.
- Use conditional statements to handle user input.
- List methods are widely used in real-world applications (e.g., to-do apps, shopping carts, inventory systems).
- Practice helps you build problem-solving skills.

## Try It Yourself

- Modify the program to allow editing a task (replace a task).
- Add an option to clear all tasks (use clear()).
- Save the tasks to a file (advanced).
- Add task numbers while displaying the list.
- Create a menu-driven to-do list with more features (e.g., mark task as completed).

Code | Practice | Grow
A Better Tomorrow

*"Consistent practice turns small codes into big dreams."* – Satyarthi